Anonymous Self-Credentials

and Application to Single-Sign-On

Jayamine Alupotha[1], Mariarosaria Barbaraci[1], Ioannis Kaklamanis[3], Abhimanyu Rawat[2], Christian Cachin[1], Fan Zhang[3]
[1] University of Bern, [2] University of Barcelona, [3] Yale University

Why do most online accounts require an email address or a phone number?

The main reason is **Sybil-Resistance**.

Sybil Resistance

“Mitigating a user appearing as many different users”

Why?

- Optimized resource-allocation

How?

- Financial Constraints, e.g., payment per account/identity
- Time Limitations, e.g., email address creations
- Relying on a trusted anchor, e.g., a trusted Identity provider

Sybil Resistance

“Mitigating a user appearing as many different users”

In other words, Sybil-Resistance is gained by *limiting* the user to a single identity or a very small number of identities.

Outcome of such “Sybil Resistance”

Data Linkability

alice@email.com

Centralized Identity provider

Learning

Marketing

Selling products

Current Solutions in the *Production*

alice@email.com

ecila
liace
aceli

Centralized Identity Issuer

Learning

Marketing

Selling products

Proposed Decentralized Solutions

ID:alice

sk_A

ecila

liace

aceli

Decentralized Identity Issuer

Learning

Marketing

Selling products

?

?

``Offered Privacy'' vs. ``User-centric Privacy''

All these solutions ask users to enjoy the privacy offered by someone else, i.e., identity provider.

Storages/APIs/DID-based Apps	Decentralized federated identity issuers	No federated identity providers?
• DIDKit • Hyperledger Aries • Veromo • uPort • IDChain • LifeID • IDChain • EverID • The Key • IdRamp • many others	• Y Zhou et al. (2019) [3] • Coconut (threshold) [2] • DIMS [1] • CanDID (2021) • PBIDM (2023) • LSTIDM (2024)	Is it even possible? How about sybil resistance? How about usability?

[1] D. Maram et al., "Candid: Can-do decentralized identity with legacy compatibility, sybil-resistance, and accountability," in Proc. IEEE Symp. Secur. Privacy, 2021, pp. 1348–1366.
[2] X. Yang and W. Li, "A zero-knowledge-proof-based digital identity management scheme in blockchain," Comput. Secur., vol. 99, 2020, Art. no. 102050.
[3] A. Sonnino, M. Al-Bassam, S. Bano, S. Meiklejohn, and G. Danezis, "Coconut: Threshold issuance selective disclosure credentials with applications to distributed ledgers," in Proc. Netw. Distrib. Syst. Secur. (NDSS) Symp., San Diego, CA, USA, Feb. 24–27, 2019. [19] Y. Zhou, T. Liu, F. Tang, F. Wang, and M. Tinashe, "A privacy-preserving authentication and key agreement scheme with deniability for IoT," Electronics, vol. 8, no. 4, 2019, Art. no. 450.

``Offered Privacy'' vs. ``User-centric Privacy''

Can we build a Sybil-resistant identity system where users gain their own privacy without relying on someone to offer privacy?

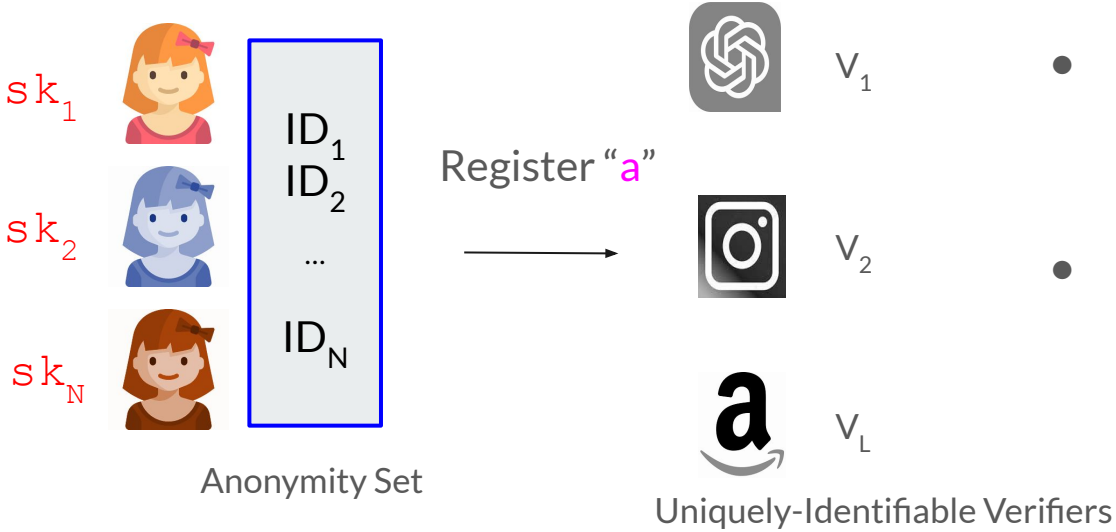
Solution: Cryptography

Next Question:

- What kind of a cryptographic protocol do we need?

Anonymous Self-Credentials (ASC)

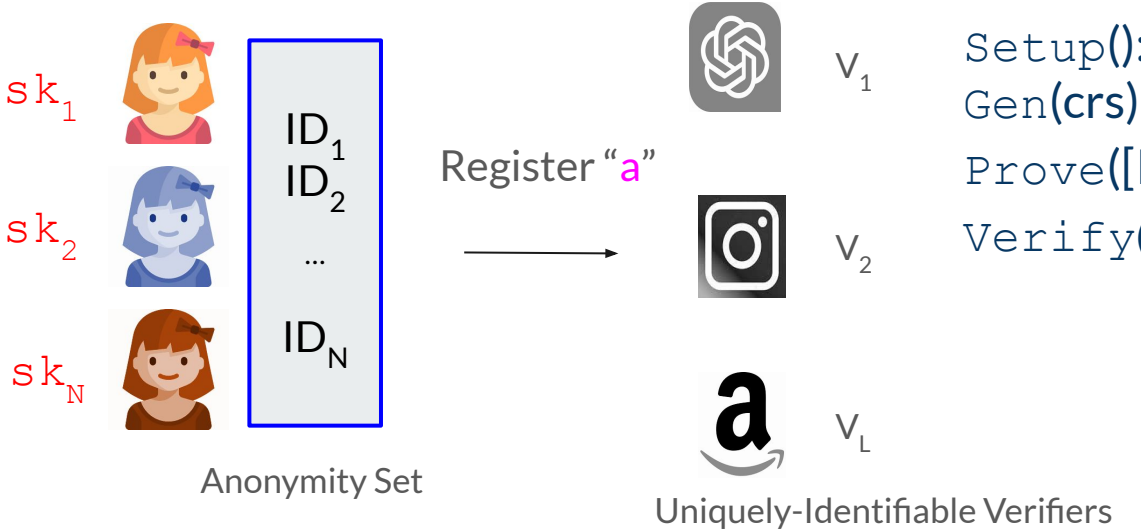
N provers with $ID_1, \ldots, ID_N$ and *L verifiers with unique names:* $V_1, \ldots, V_L$



- Prove that pseudonym "a" came from a prover in the anonymity set without revealing the identity.
- Any two pseudonyms from the same prover ID_j for the same verifier V_i should be linkable.

Anonymous Self-Credentials (ASC)

N provers with $ID_1, \ldots, ID_N$ and *L verifiers with unique names:* $V_1, \ldots, V_L$



`Setup`(): crs (including verifier names)

`Gen`(crs) : (ID, sk)

`Prove`([$ID_1, \ldots, ID_N$], sk_j, a, V_i): (π^i_j, nul)

`Verify`([$ID_1, \ldots, ID_N$], a, V_i, π^i_j, nul): T/F

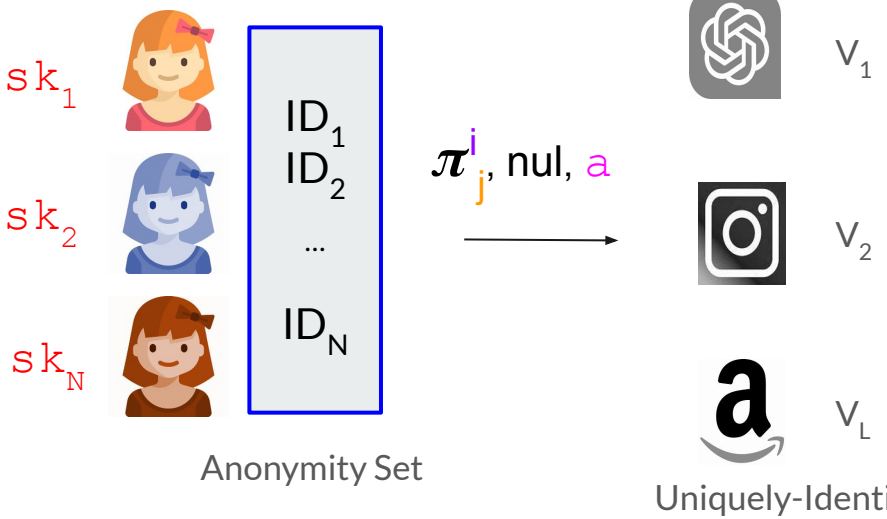
π^i_j: Proof for V_i created by ID_j

nul: Nullifier – unique to (sk_j, V_i)

a: Pseudonym

Anonymous Self-Credentials (ASC)

N provers with $ID_1, \ldots, ID_N$ and *L verifiers with unique names:* $V_1, \ldots, V_L$



`Setup`(): crs (including verifier names)

`Gen`(crs) : (ID, sk)

`Prove`([ID_1,..,ID_N], sk_j, a, V_i): (π^i_j, nul)

`Verify`([ID_1,..,ID_N], a, V_i, π^i_j, nul): T/F

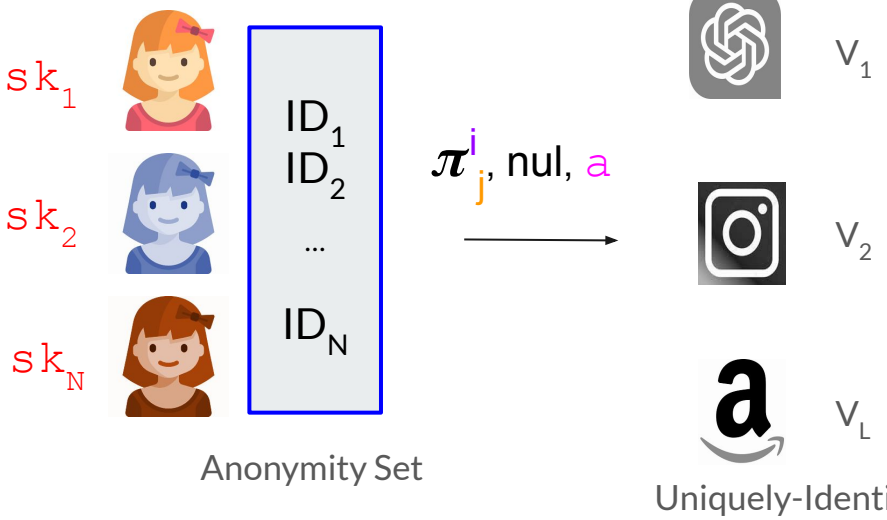
π^i_j: Proof for V_i created by ID_j

nul: Nullifier

a: Pseudonym

Anonymous Self-Credentials (ASC)

N provers with $ID_1, \ldots, ID_N$ and *L verifiers with unique names:* $V_1, \ldots, V_L$



- Each verifier locally saves nullifiers used to register pseudonyms.
- A verifier registers a new "a" if its nullifier was not used to register a previous pseudonym.

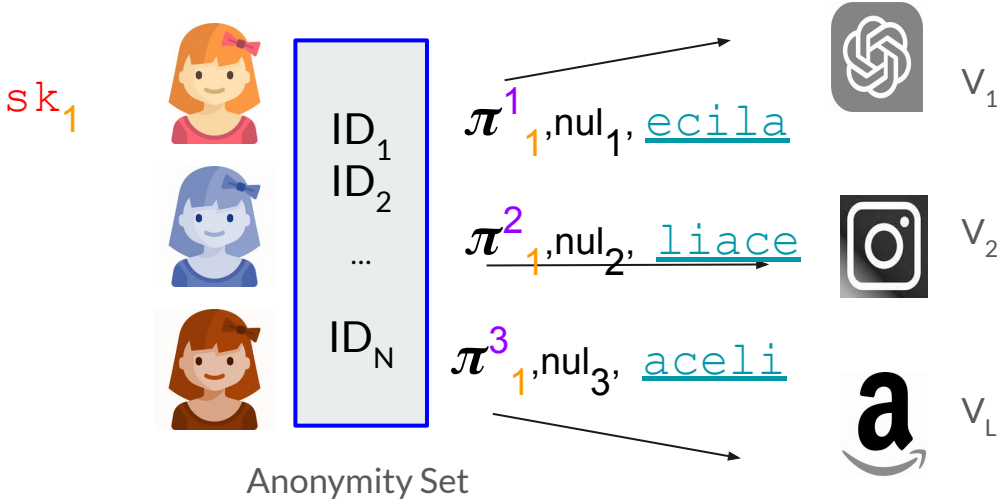
π^i_j: Proof for V_i created by ID_j
nul: Nullifier
a: Pseudonym

Anonymous Self-Credentials: Next

1) Correctness
2) Robustness
3) Unforgeability
4) Sybil-Resistance
5) Anonymity
6) Unlinkability

Anonymous Self-Credentials: Correctness (part 1)

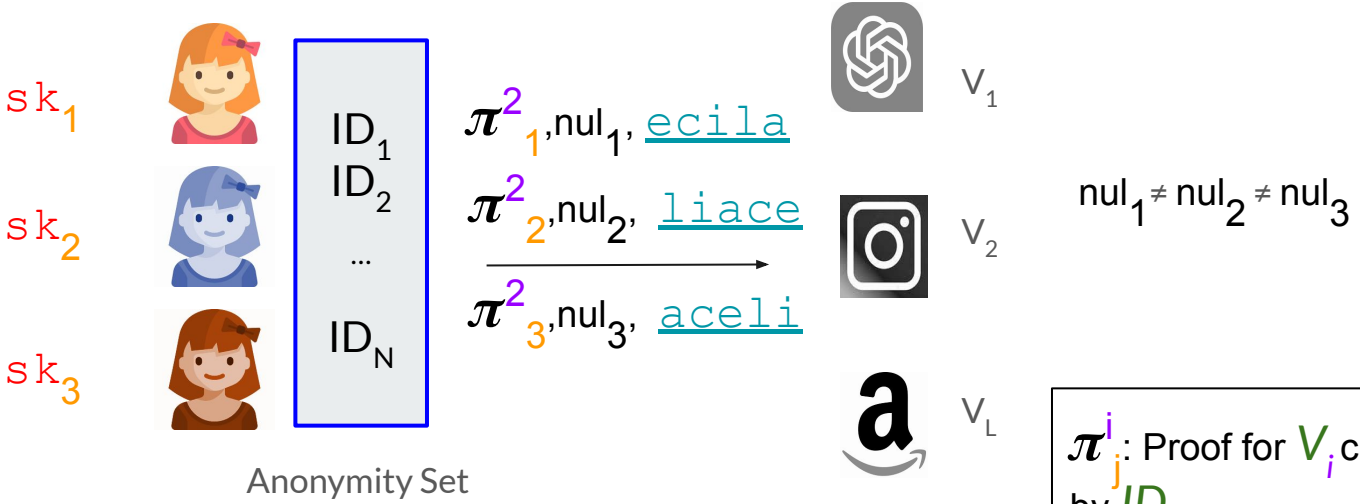
Any honest prover should be able to **self-generate** valid proofs and nullifiers for *any pseudonyms* for *all verifiers*.



π^i_j: Proof for V_i created by ID_j

Anonymous Self-Credentials: Correctness (part 2)

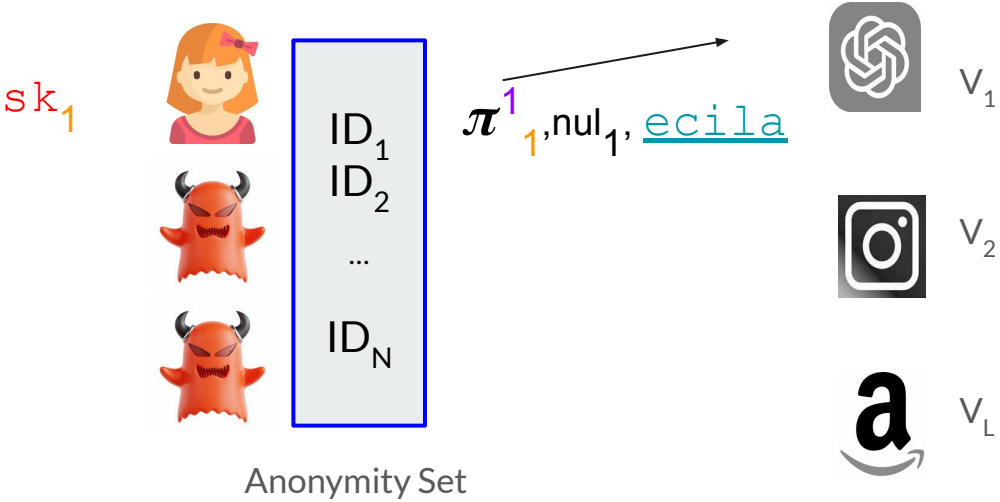
Different honest provers' nullifiers should be different for *the same verifier* for *any pseudonyms*.



π^i_j: Proof for V_i created by ID_j

Anonymous Self-Credentials: Robustness

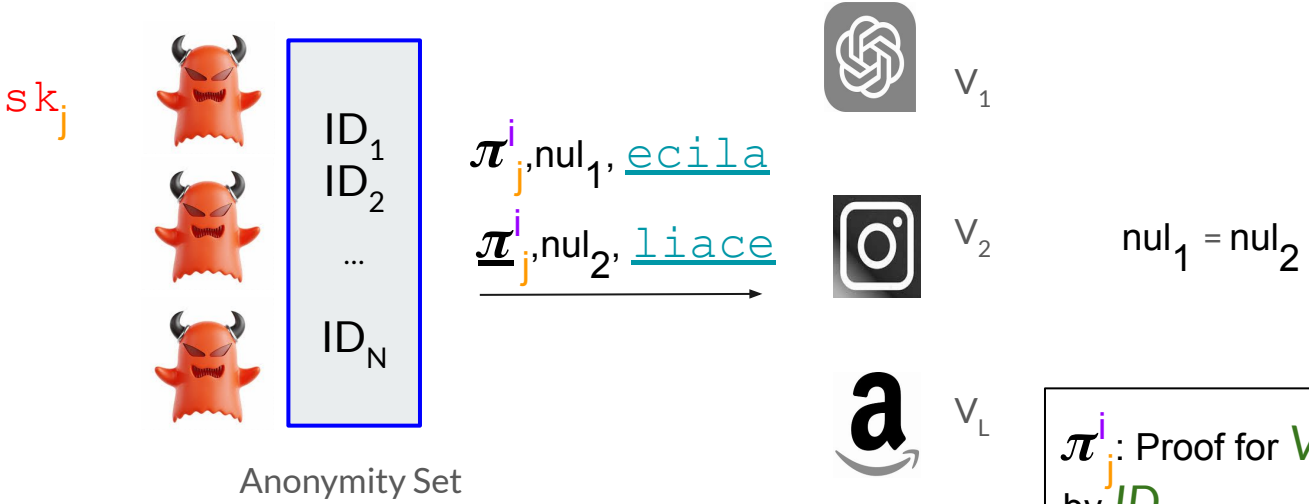
Malicious provers cannot hinder the pseudonym registration of honest provers.



π^i_j: Proof for V_i created by ID_j

Anonymous Self-Credentials: Sybil Resistance

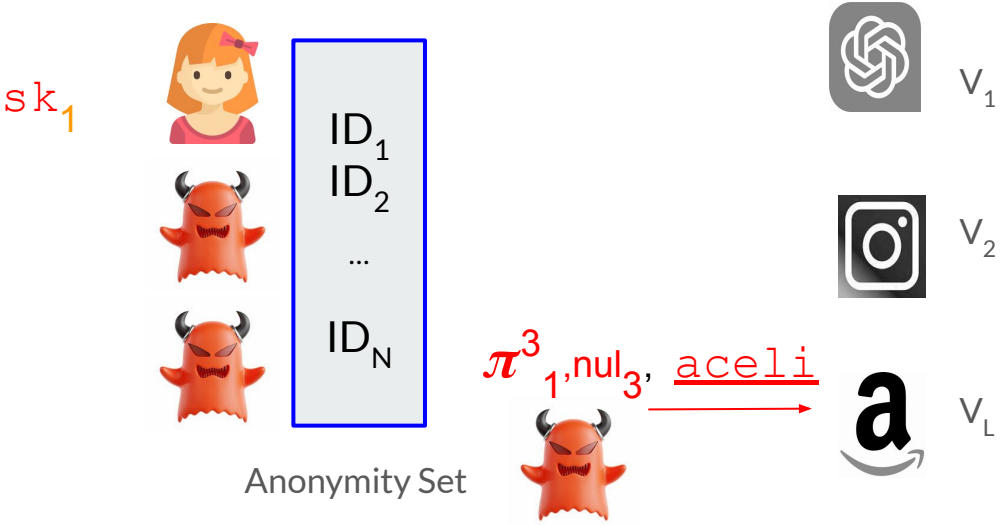
Any corrupted prover of ID_j cannot generate *different nullifiers* for the same verifier V_i for any pseudonyms.



π^i_j: Proof for V_i created by ID_j

Anonymous Self-Credentials: Unforgeability (part 1)

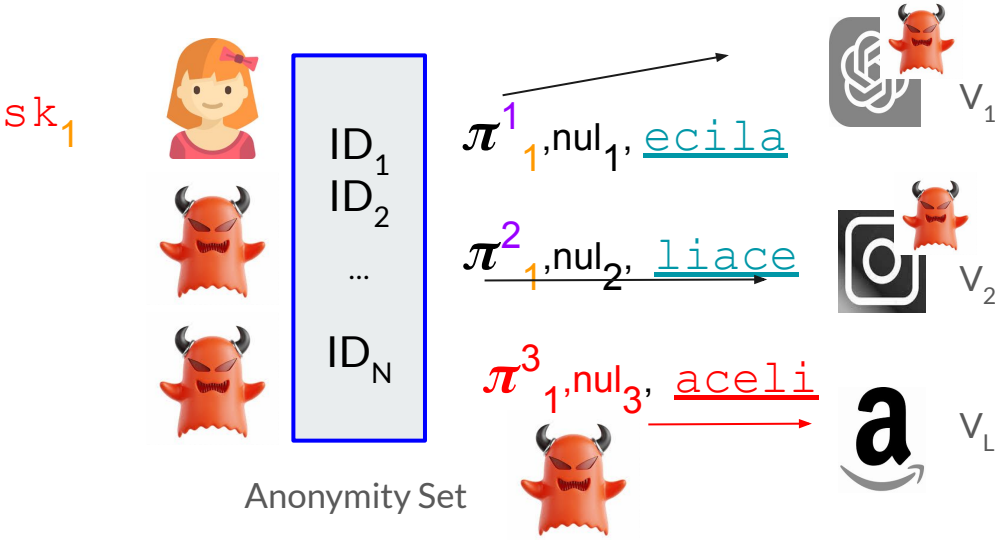
An adversary who controls all provers, *except one honest prover* cannot forge a proof for a honest verifier.



π^i_j: Proof for V_i created by ID_j

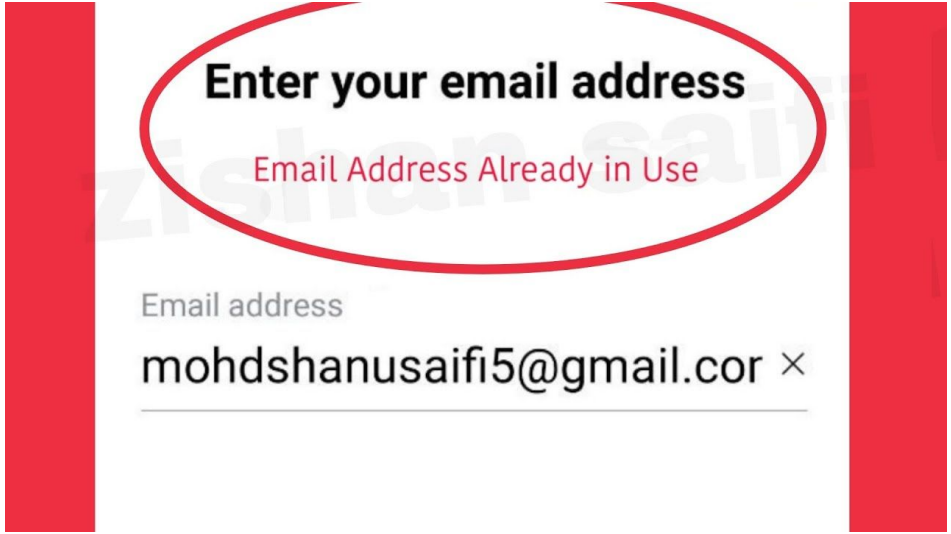
Anonymous Self-Credentials: Unforgeability (part 2)

An adversary who controls all provers, *except one honest prover* cannot forge a proof for a *honest verifier* even with proofs of corrupted verifiers.



π^i_j: Proof for V_i created by ID_j

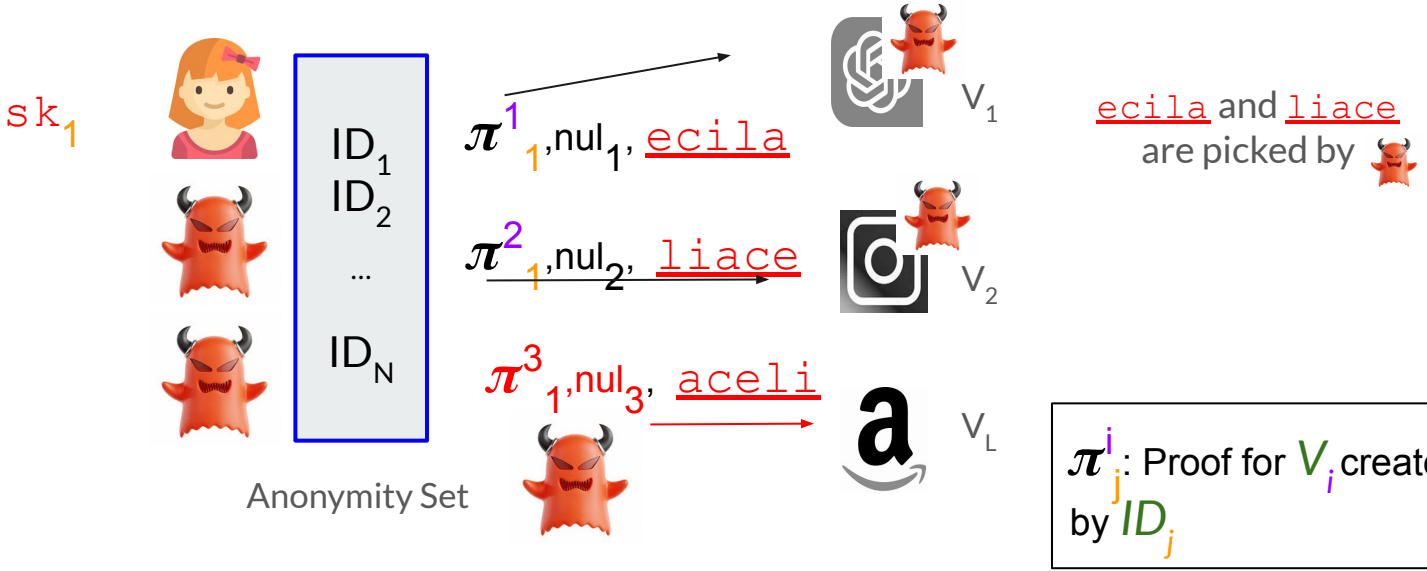
Anonymous Self-Credentials: Unforgeability (part 3)



Verifiers have the authority to determine which pseudonyms the provers are allowed to choose!

Anonymous Self-Credentials: Unforgeability (part 3)

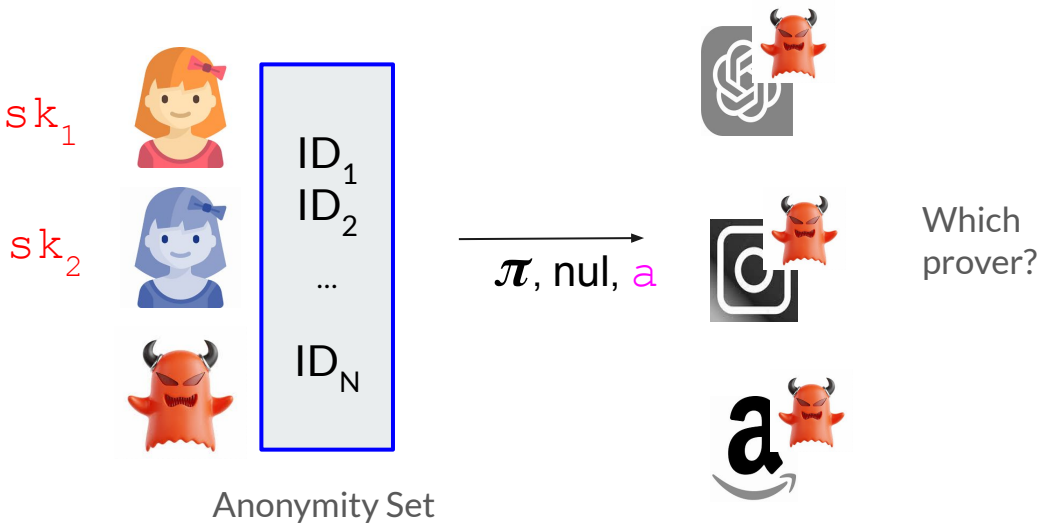
An adversary that controls all provers, *except one honest prover* cannot forge a proof for a *fresh pseudonym* even with proofs of corrupted verifiers for adversary picked pseudonyms.



π^i_j: Proof for V_i created by ID_j

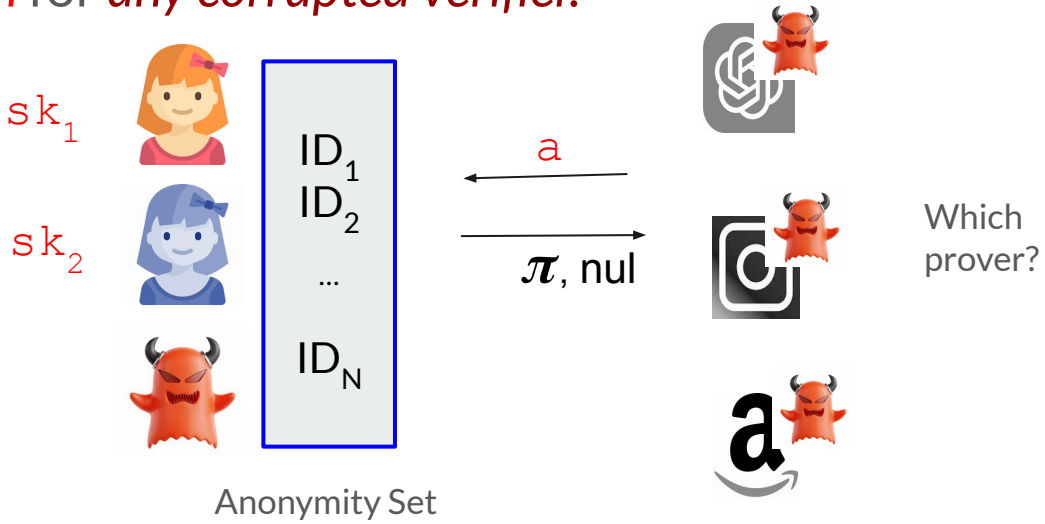
Anonymous Self-Credentials: Anonymity (part 1)

An adversary that controls all provers, *except two honest provers*, cannot identify *which prover* created the proof for *any corrupted verifier.*



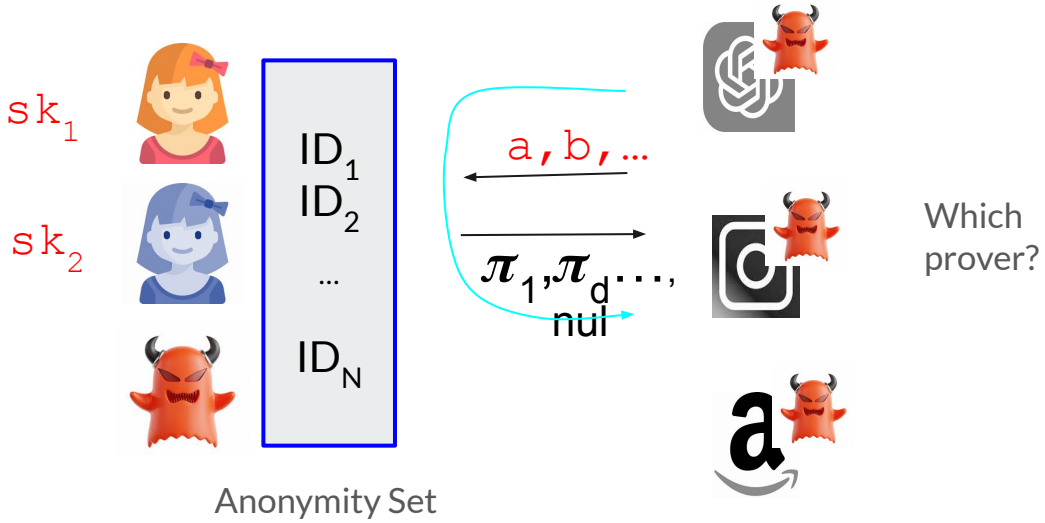
Anonymous Self-Credentials: Anonymity (part 2)

An adversary that controls all provers, *except two honest provers*, cannot identify *which prover* created *the proof for a adversary-picked pseudonym* for *any corrupted verifier.*



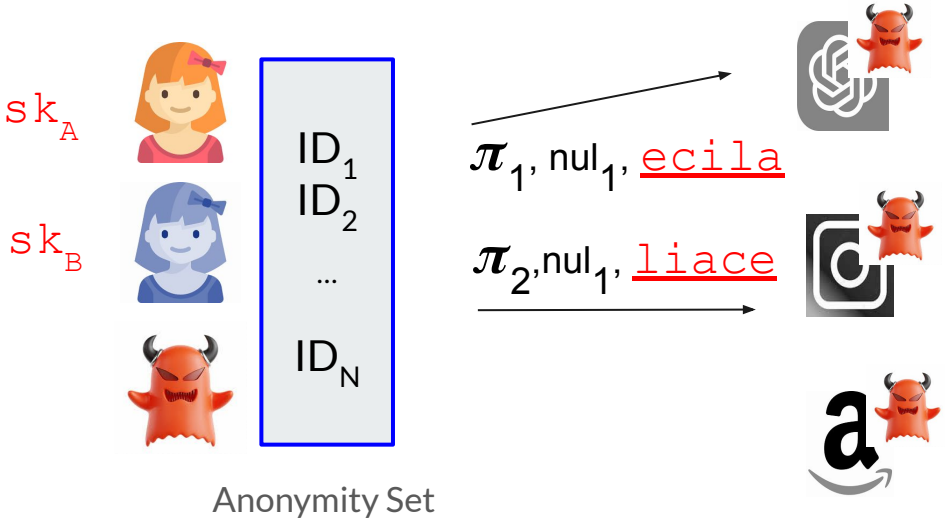
Anonymous Self-Credentials: Anonymity (part 3)

An adversary that controls all provers, *except two honest provers*, cannot identify *which prover* created *the proofs for **multiple** adversary-picked pseudonyms* for *any corrupted verifier.*



Anonymous Self-Credentials: Unlinkability

An adversary that controls all provers, *except two honest provers* cannot identify whether *two proofs* for *any two verifiers* were generated by *the same prover or two provers*.



Same prover or two different provers?

Anonymous Self-Credentials: Recap

1) Correctness
2) Robustness
3) Unforgeability
4) Sybil-Resistance
5) Anonymity
6) Unlinkability

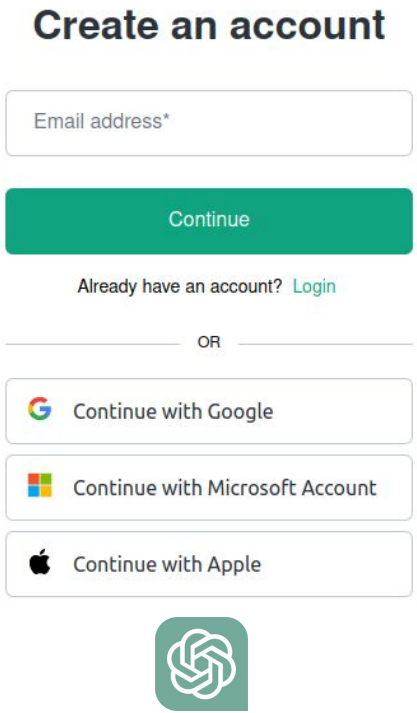
Anonymous Self-Credentials: Potential Approaches

The protocols that realize anonymous self-credentials:

1) traceable ring signatures [1],
2) zk-SNARK circuit for anonymous messaging [2], and
3) Double Blind Proof of Existence [3].

[1] Fujisaki, E., & Suzuki, K. (2007, April). Traceable ring signature. In *International Workshop on Public Key Cryptography* (pp. 181-200). Berlin, Heidelberg: Springer Berlin Heidelberg.
[2] https://github.com/semaphore-protocol/semaphore
[3] Alupotha, J. (2023). Double-blind proof of existence for decentralized identities. *IEEe Access*, *11*, 132180-132195.

Single-Sign-On (SSO)



Single-Sign-On from Anonymous Self-Credentials

$[V_1, \ldots, V_N]$

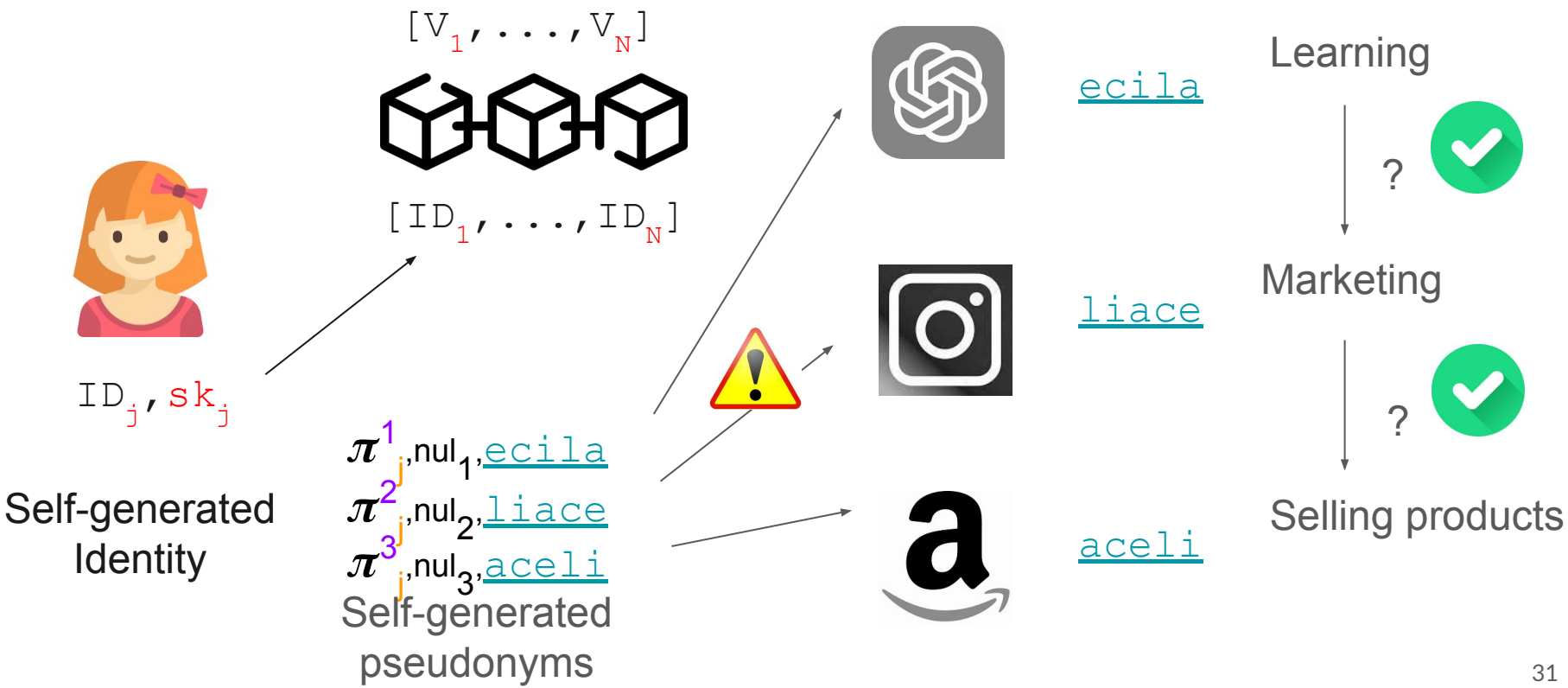
$[ID_1, \ldots, ID_N]$

ID_j, sk_j

Immutable Identity Registry with a Sybil Resistant onboarding

Self-generated Identities

Single-Sign-On from ASC (Naive Solution)



User-Issued Unlinkable SSO : U2SSO/ “you too” SSO

ID_j, sk_j

Self-generated Identity

$[ID_1, ..., ID_N]$

$\pi^1_j, nul_1, ck1$

$\pi^2_j, nul_2, ck2$

$\pi^3_j, nul_3, ck3$

Self-generated Child-Credentials

ck1

ck2

ck3

U2SSO Authentication

ID_j, sk_j

Self-generated Identity

$[ID_1, ..., ID_N]$

Prove the Ownership of ck3

ck1

ck2

ck3

U2SSO Implementations

SRS-U2SSO:

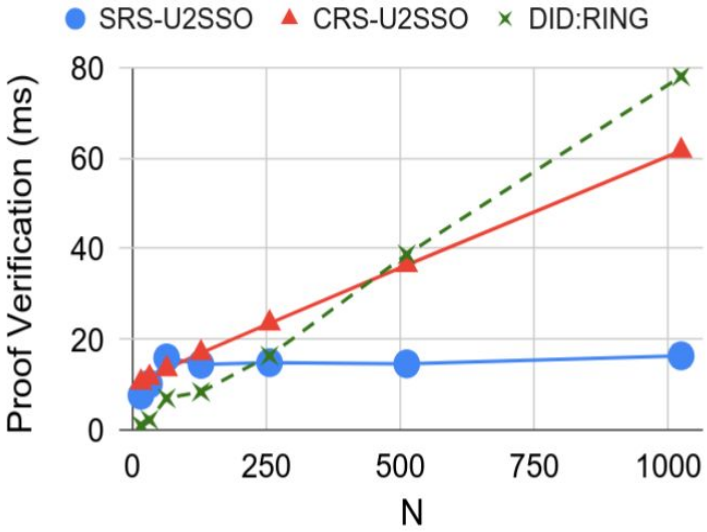
1) ASC from zk-SNARK circuit
2) ID size: 96 bytes
3) Proof Size: 328 bytes for any (N, L)
4) Simulated with BLS signature key pairs as child-credentials
5) Trusted setup required

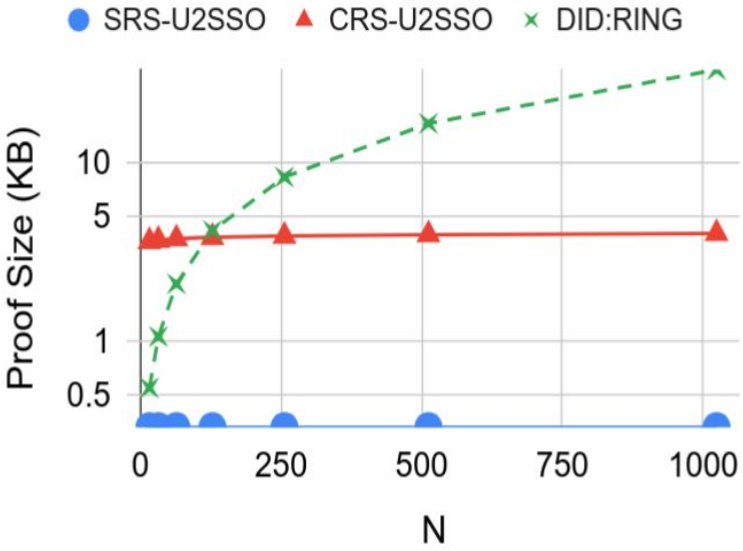
CRS-U2SSO:

1) ASC from Double Blind Proof of Existence
2) ID size: 33 bytes
3) Proof Size: O(log N + L)
4) Simulated with Schnorr signature key pairs as child-credentials
5) Transparent setup

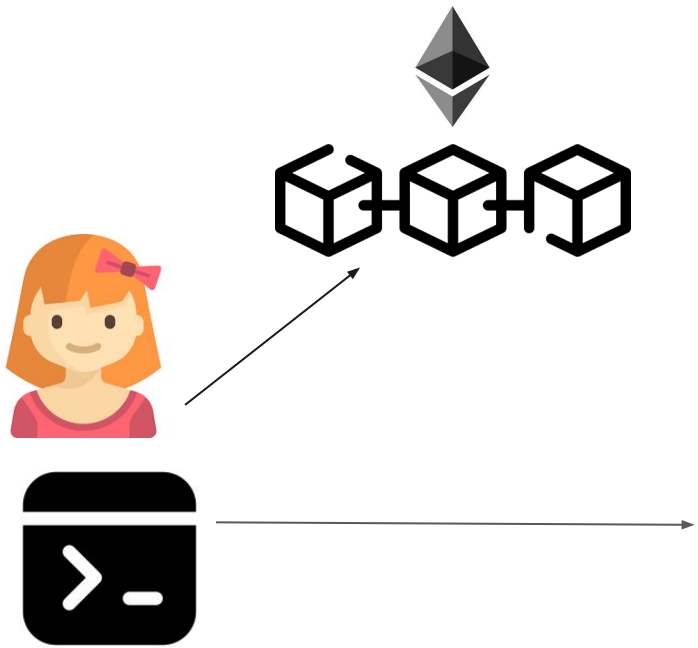
?-U2SSO:

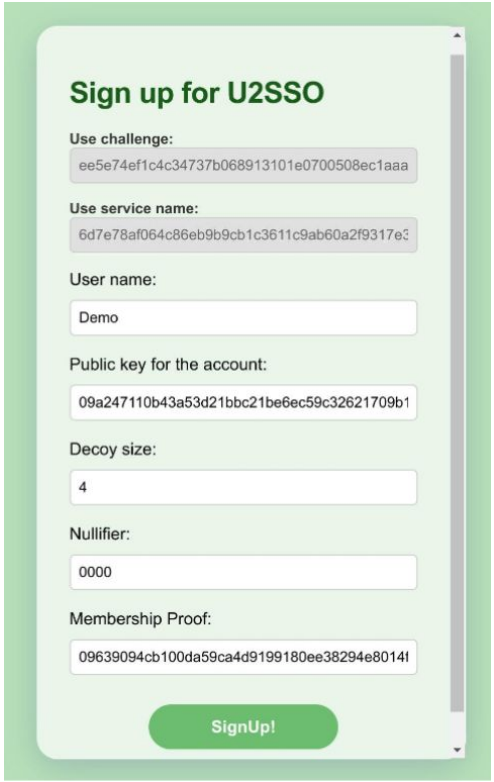
U2SSO Implementations

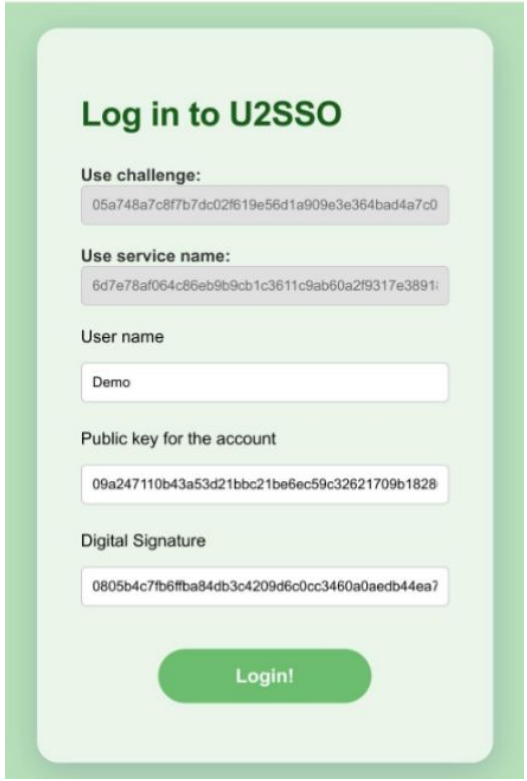




U2SSO Implementations







Federated SSO vs. U2SSO

Federated SSO:

1) Frequent interactions with the federated identity provider.
2) Child credential scheme is chosen by the identity provider
3) Offered Privacy

U2SSO:

1) Minimal interactions with Identity Registry, e.g., no interactions during the authentication.
2) Plug-and-play ASC and Child credentials
3) **User-centric privacy**

End

https://github.com/BoquilaID/U2SSO

Thank you!